HPC Laboratory – Experiment No. 9

Introduction to CUDA Programming: Fundamentals of GPU Computing

Name: Aditi Mittal

PRN: 23610067 | Batch: B1

Final Year B.Tech CSE

Walchand College of Engineering

Google Colab Notebook:
https://colab.research.google.com/drive/1jDc_U5Q1tYqDomfXlIxHywb3cRJArA8m?usp=sharing

1. Aim

To introduce students to CUDA programming by implementing basic GPU programs that demonstrate thread management, memory operations, and performance comparison between CPU and GPU execution.

2. Objectives

1. Understand the CUDA programming model including threads, blocks, and grids.
2. Write and compile basic CUDA programs using nvcc.
3. Implement parallel vector operations on the GPU.
4. Measure and compare CPU vs GPU execution times using CUDA events.
5. Explore CUDA memory management including explicit Host→Device and Device→Host transfers.
6. Investigate the effect of block size on kernel performance.
7. Compare sequential, OpenMP (multi-threaded CPU), and CUDA execution.
8. Apply CUDA to a practical squaring operation and verify correctness.

3. System Configuration (Google Colab – Tesla T4)

Obtained from nvidia-smi and cudaGetDeviceProperties (Task 1 output):

Property	Value
GPU Model	NVIDIA Tesla T4
Driver Version	580.82.07
CUDA Version	13.0
Total Global Memory	14,913 MB (~15 GB)
Compute Capability	7.5 (Turing architecture)
Streaming Multiprocessors	40 SMs
Max Threads per Block	1,024
Max Grid Dimensions X	2,147,483,647
Warp Size	32
Memory Bus Width	256-bit
L2 Cache Size	4,096 KB
Shared Memory per Block	48 KB
GPU Temperature	35°C

4. Theory: CUDA Programming Model

4.1 What is CUDA?

CUDA (Compute Unified Device Architecture) is NVIDIA's parallel computing platform that allows developers to write programs executed on the GPU. Programs use thousands of parallel threads to perform computations simultaneously, offering massive speedups for data-parallel workloads.

4.2 Host vs Device

Concept	Meaning
Host	The CPU and its main memory (RAM)
Device	The GPU and its dedicated VRAM
Kernel	A function that runs on the GPU, executed by many threads in parallel

4.3 Thread Hierarchy

CUDA organizes parallel execution in a three-level hierarchy: Threads → Blocks → Grid. A thread is the smallest unit of execution. A block groups up to 1,024 threads that can share fast shared memory and synchronize via __syncthreads(). A grid contains all blocks for one kernel launch.

Global Thread ID formula (1D):

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
```

4.4 CUDA Built-in Variables

Variable	Meaning
threadIdx.x	Thread index within its block (0 to blockDim.x−1)
blockIdx.x	Block index within the grid (0 to gridDim.x−1)
blockDim.x	Number of threads per block
gridDim.x	Number of blocks in the grid

4.5 CUDA Memory Types

Memory Type	Location	Speed	Scope
Global memory	GPU VRAM	~900 GB/s	All threads + host
Shared memory	Per-block SRAM	~19 TB/s	Same block only
Registers	Per-thread	Fastest	Single thread
Constant memory	GPU VRAM (cached)	Fast	All threads (read-only)

4.6 CUDA Memory Management API

```
cudaMalloc(&d_ptr, size);                                   // Allocate device memory
cudaMemcpy(dst, src, size, cudaMemcpyHostToDevice);         // Copy host → device
cudaMemcpy(dst, src, size, cudaMemcpyDeviceToHost);         // Copy device → host
cudaFree(d_ptr);                                            // Free device memory
```

4.7 CUDA Timing with Events

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
// ... kernel launch ...
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float ms;
cudaEventElapsedTime(&ms, start, stop);  // elapsed time in milliseconds
```

cudaEvent_t provides GPU-side hardware timestamps accurate to microseconds, independent of CPU scheduling jitter.

4.8 Kernel Launch Syntax

```
// numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;
kernel<<<numBlocks, threadsPerBlock>>>(arg1, arg2, ...);
```

5. Tasks and Outputs

Task 1: GPU Device Information

Objective: Query and display properties of the available CUDA device using cudaGetDeviceProperties.

Code:

```
#include <stdio.h>
#include <cuda_runtime.h>

int main() {
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);
    printf("Number of CUDA devices: %d\n", deviceCount);

    cudaDeviceProp prop;
    cudaGetDeviceProperties(&prop, 0);
    printf("Device: %s\n", prop.name);
    printf("Compute Capability: %d.%d\n", prop.major, prop.minor);
    printf("Total Global Memory: %.0f MB\n", prop.totalGlobalMem/(1024.0*1024.0));
    printf("Multiprocessors: %d\n", prop.multiProcessorCount);
    printf("Max Threads per Block: %d\n", prop.maxThreadsPerBlock);
    printf("Warp Size: %d\n", prop.warpSize);
    printf("Shared Memory per Block: %zu KB\n", prop.sharedMemPerBlock/1024);
    return 0;
}
```

Output:

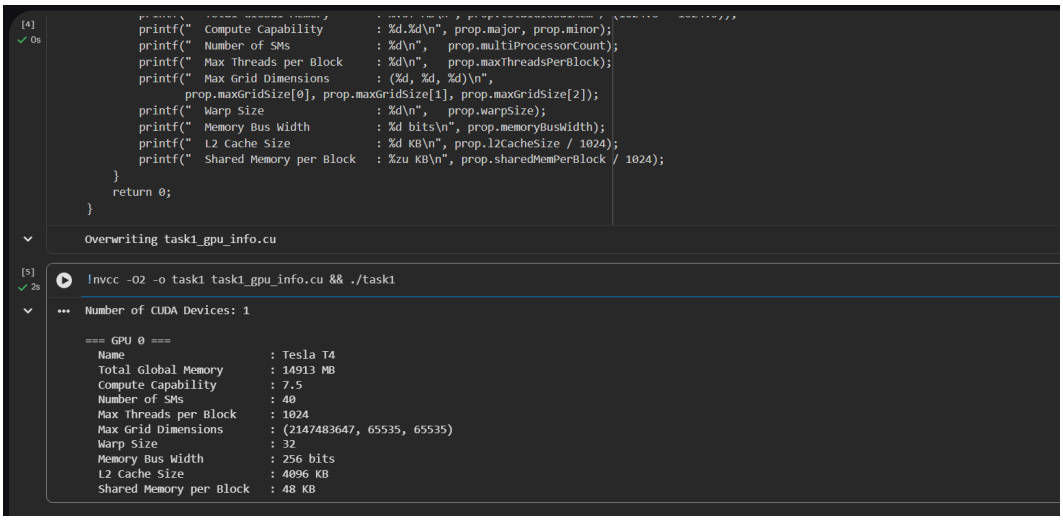


Figure 1: Task 1 – GPU Device Properties (Tesla T4 on Colab)

Note: clockRate and memoryClockRate fields were removed in CUDA 12.x. These lines were removed from our code to compile correctly under CUDA 13.0 on Colab.

Task 2: Hello from GPU – Thread and Block Configurations

Objective: Launch a simple "Hello" kernel with four different grid/block configurations.

Code (kernel function):

```
__global__ void hello_gpu() {
    printf("Hello from GPU! Block %d, Thread %d\n", blockIdx.x, threadIdx.x);
}

// Four launches in main():
hello_gpu<<<1, 8>>>();   cudaDeviceSynchronize();
hello_gpu<<<1, 16>>>();  cudaDeviceSynchronize();
hello_gpu<<<2, 8>>>();   cudaDeviceSynchronize();
hello_gpu<<<4, 8>>>();   cudaDeviceSynchronize();
```

Output:

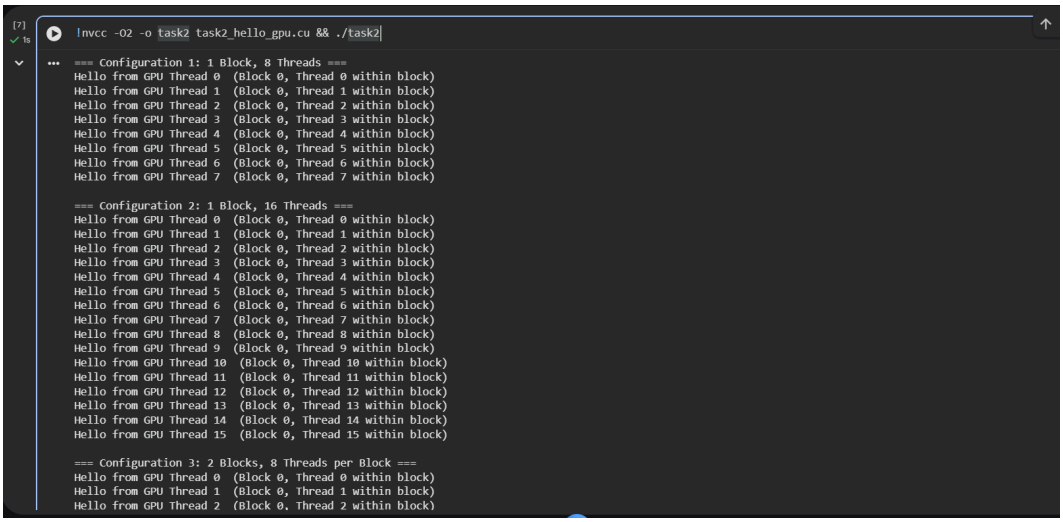


Figure 2: Task 2 – Hello GPU with four configurations

cudaDeviceSynchronize() ensures the CPU waits for all GPU threads to finish printing before proceeding to the next configuration header.

Task 3: Thread and Block ID Visualization

Objective: Visualize how threadIdx, blockIdx, and global thread ID relate in a 2D grid layout.

Code (kernel):

```
__global__ void print_ids() {
    int global_id = blockIdx.x * blockDim.x + threadIdx.x;
    printf("Block %d | Thread %d | Global ID %d\n",
           blockIdx.x, threadIdx.x, global_id);
}
```

Output:

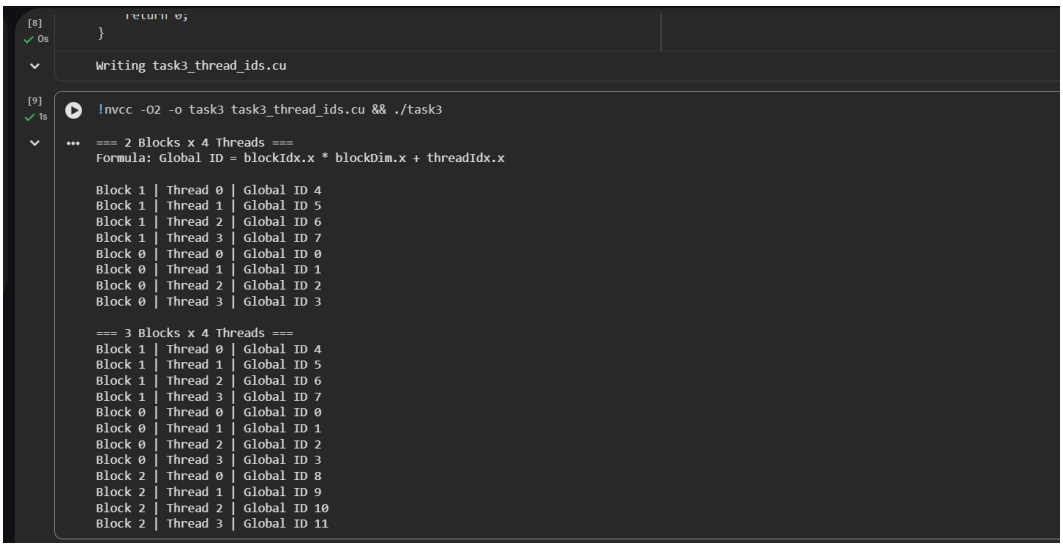


Figure 3: Task 3 – Thread and Block ID visualization

Key observation: Blocks print in non-sequential order (e.g., Block 2 before Block 0). This is by design — the GPU scheduler assigns blocks to SMs independently. There is no guaranteed execution order between blocks in CUDA.

Task 4: Parallel Vector Addition with CPU vs GPU Timing

Objective: Add two vectors (N=1,000,000) in parallel and compare CPU vs GPU kernel time.

CUDA Kernel:

```
#define N 1000000
#define THREADS_PER_BLOCK 256

__global__ void vector_add(float *a, float *b, float *c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n)
        c[idx] = a[idx] + b[idx];    // Each thread adds one element
}
```

Launch configuration:

```
int blocks = (N + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;  // = 3907
vector_add<<<blocks, THREADS_PER_BLOCK>>>(d_a, d_b, d_c, N);
```

Output:

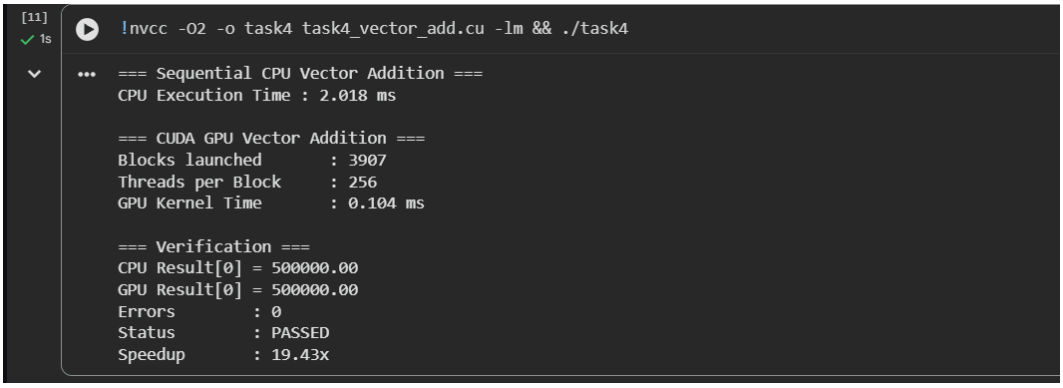


Figure 4: Task 4 – Vector Addition Performance

Metric	Value
CPU Time	2.018 ms
GPU Kernel Time	0.104 ms
Speedup	19.43×
Grid	3,907 blocks × 256 threads
Correctness	PASSED

Task 5: Explicit CUDA Memory Management

Objective: Measure individual timings of H→D transfer, kernel, and D→H transfer.

Key code for timing each phase:

```
cudaEventRecord(e0);
cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);    // H→D
cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);
cudaEventRecord(e1);
vector_add<<<blocks, THREADS_PER_BLOCK>>>(d_a, d_b, d_c, N);  // Kernel
cudaEventRecord(e2);
cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);    // D→H
cudaEventRecord(e3);
cudaEventSynchronize(e3);
```

Output:

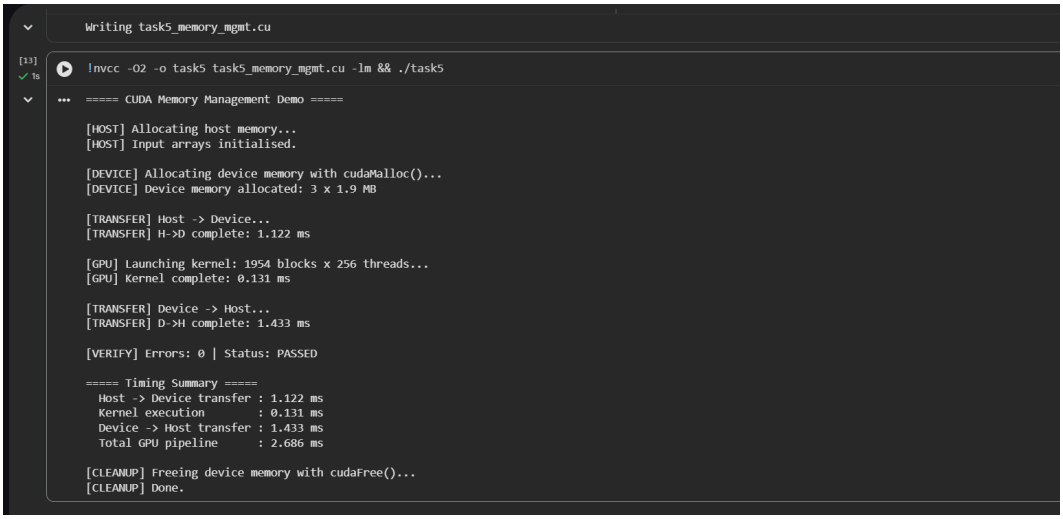


Figure 5: Task 5 – Memory Management Timing Breakdown

Operation	Time (ms)
Host → Device Transfer	1.122
Kernel Execution	0.131
Device → Host Transfer	1.433
Total	2.686

Memory transfers (2.555 ms combined) take 19× longer than the compute kernel (0.131 ms). This illustrates the critical importance of minimizing data movement between CPU and GPU.

Task 6: Block Size Benchmark

Objective: Measure how different thread-per-block values affect kernel time (N=1,000,000).

Output:

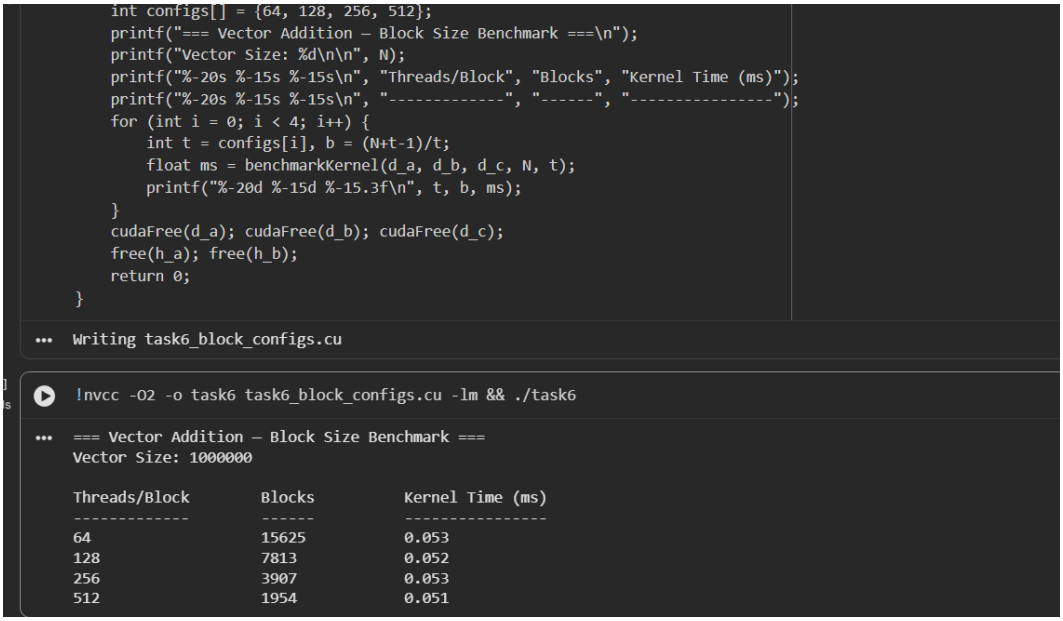
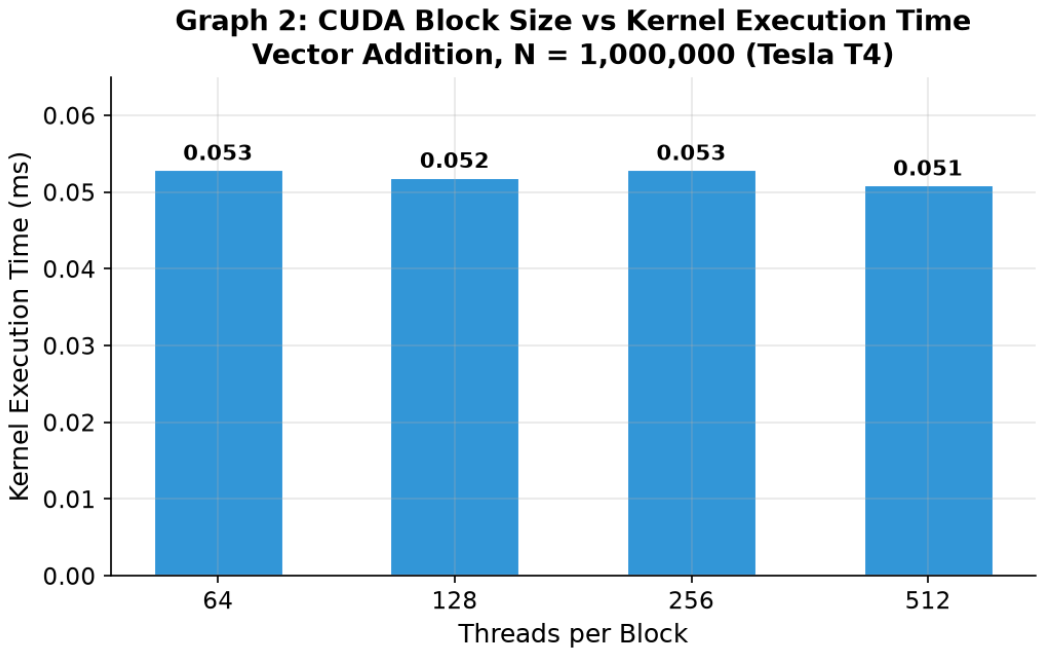


Figure 6: Task 6 – Block Size Benchmark Output

Threads per Block	Kernel Time (ms)
64	0.053
128	0.052
256	0.053
512	0.051



Graph 2: CUDA Block Size vs Kernel Execution Time (N=1,000,000, Tesla T4)

Task 7: Sequential vs OpenMP vs CUDA Comparison

Objective: Compare vector addition across three implementations as size scales from 10^5 to 10^8.

OpenMP kernel:

```
void omp_add(float *a, float *b, float *c, int n) {
    #pragma omp parallel for
    for (int i = 0; i < n; i++) c[i] = a[i] + b[i];
}
```

CUDA kernel (same as Tasks 4/5/6):

```
__global__ void cuda_add(float *a, float *b, float *c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) c[idx] = a[idx] + b[idx];
}
```

Output:

```
        printf("OpenMP threads: %d\n\n", omp_get_max_threads());
        long sizes[] = {100000L, 1000000L, 10000000L, 100000000L};
        for (int i=0; i<4; i++) runBenchmark(sizes[i]);
        return 0;
    }

Writing task7_comparison.cu

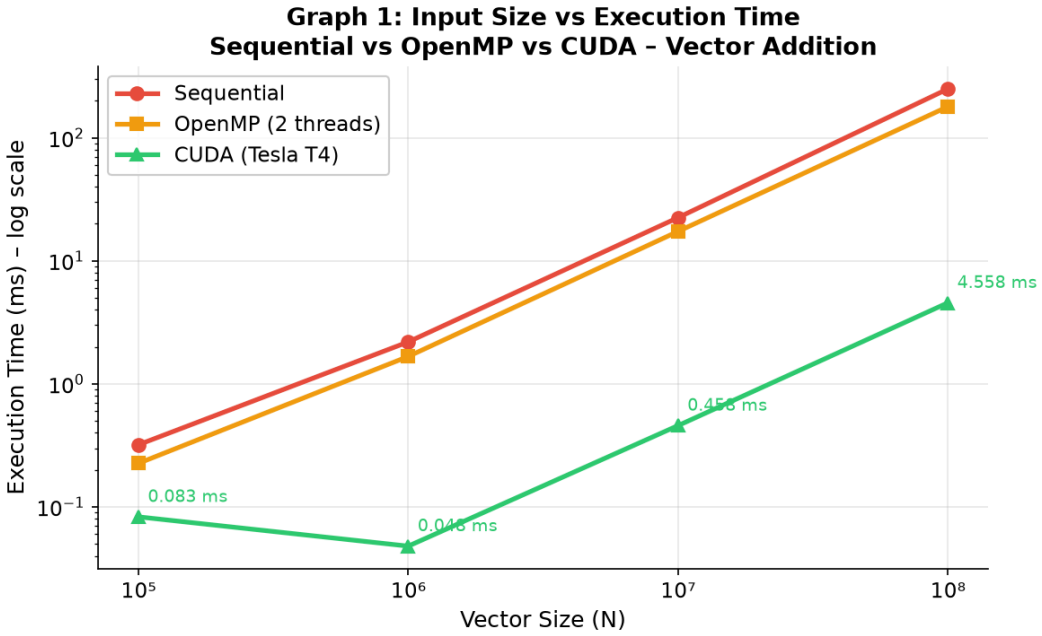
!nvcc -O2 -Xcompiler -fopenmp -o task7 task7_comparison.cu -lgomp -lm && ./task7

=== Sequential vs OpenMP vs CUDA — Vector Addition ===
OpenMP threads: 2

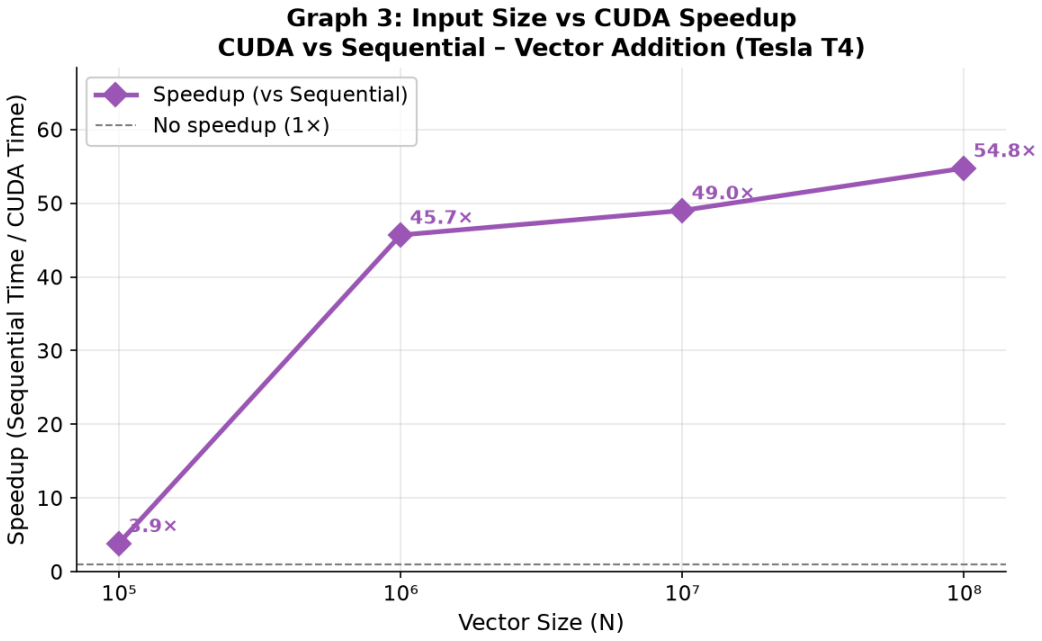
  N =   100000 | Seq:    0.320 ms | OMP:    0.226 ms | CUDA:    0.083 ms | Err: 0
  N =  1000000 | Seq:    2.193 ms | OMP:    1.670 ms | CUDA:    0.048 ms | Err: 0
  N = 10000000 | Seq:   22.447 ms | OMP:   17.343 ms | CUDA:    0.458 ms | Err: 0
  N = 100000000 | Seq:  249.584 ms | OMP:  179.032 ms | CUDA:    4.558 ms | Err: 0
```

Figure 7: Task 7 – Sequential vs OpenMP vs CUDA

N	Sequential (ms)	OpenMP (ms)	CUDA (ms)	CUDA Speedup
100,000	0.320	0.226	0.083	3.86×
1,000,000	2.193	1.670	0.048	45.69×
10,000,000	22.447	17.343	0.458	49.01×
100,000,000	249.584	179.032	4.558	54.76×



Graph 1: Input Size vs Execution Time – Sequential vs OpenMP vs CUDA (log scale)



Graph 3: CUDA Speedup vs Input Size (relative to Sequential)

Task 8: Array Squaring

Objective: Apply GPU parallelism to element-wise squaring and verify against CPU.

CUDA Kernel:

```
__global__ void square_array(float *in, float *out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) out[idx] = in[idx] * in[idx];
}
```

Output:

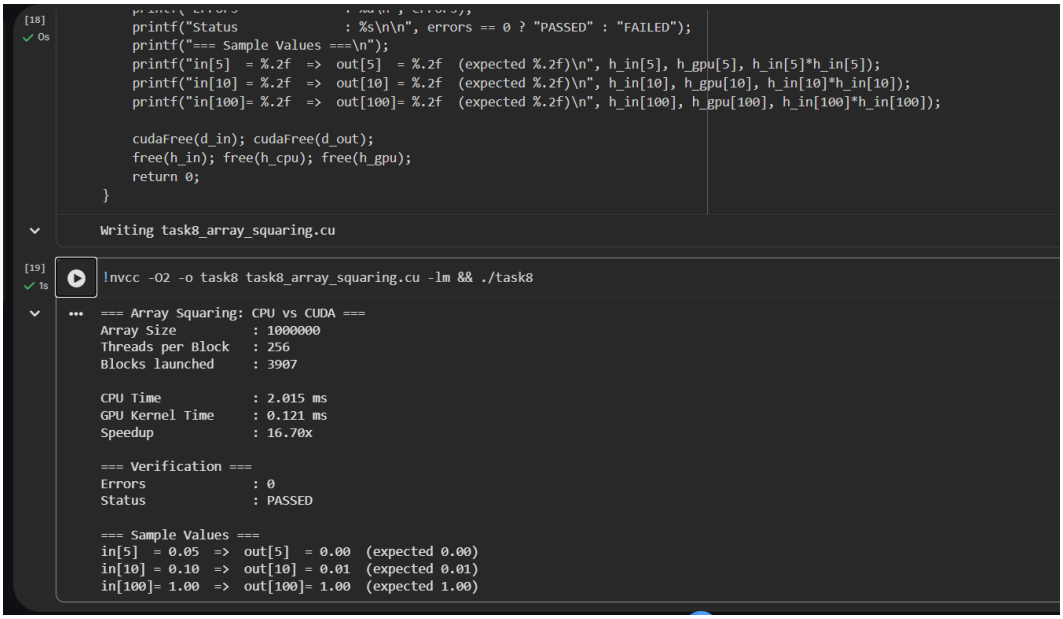


Figure 8: Task 8 – Array Squaring Performance

Metric	Value
N	1,000,000
Threads per Block	256
Blocks	3,907
CPU Time	2.015 ms
GPU Kernel Time	0.121 ms
Speedup	16.70×
Correctness	PASSED

6. Performance Summary

6.1 CPU vs GPU Kernel Speedup

Task	Operation	N	CPU (ms)	GPU Kernel (ms)	Speedup
Task 4	Vector Addition	1,000,000	2.018	0.104	19.43×
Task 7	Vector Addition	100,000,000	249.584	4.558	54.76×
Task 8	Array Squaring	1,000,000	2.015	0.121	16.70×

6.2 Memory Transfer Breakdown (Task 5, N=500,000)

Phase	Time (ms)	Fraction of Total
Host → Device	1.122	41.8%
Kernel Exec	0.131	4.9%
Device → Host	1.433	53.4%
Total	2.686	100%

Transfers consume 95% of total GPU pipeline time for this workload — compute is not the bottleneck.

7. Questions and Answers

Q1. What is the difference between CPU and GPU architecture?

CPU architecture is optimized for latency — it has a small number of powerful cores (typically 4–32) with large caches and out-of-order execution, designed to run sequential code fast. GPU architecture is optimized for throughput — the Tesla T4 has 40 SMs (2,560 CUDA cores) designed to run thousands of threads simultaneously on simple arithmetic. CPUs minimize latency per task; GPUs maximize total work per second.

Q2. What is a CUDA thread and how is it different from a CPU thread?

A CUDA thread is an extremely lightweight execution unit running one kernel iteration. Unlike CPU threads (OS-managed, heavyweight, context-switch overhead), CUDA threads are managed entirely in hardware. An SM can hold 1,024 active threads and switches between groups of 32 (warps) in zero cycles, hiding memory latency through parallelism.

Q3. Explain the concept of thread blocks and grids in CUDA.

A thread block is a group of up to 1,024 threads executing on a single SM. Threads in a block share 48 KB of fast shared memory and synchronize via __syncthreads(). A grid is the entire collection of blocks for one kernel launch. The programmer specifies numBlocks and threadsPerBlock; the GPU scheduler assigns blocks to SMs automatically.

Q4. What is the role of blockIdx.x, threadIdx.x, and blockDim.x?

threadIdx.x: thread index within its block (0 to blockDim.x−1). blockIdx.x: block index within the grid (0 to gridDim.x−1). blockDim.x: total threads per block (set at launch). Combined: int global_id = blockIdx.x * blockDim.x + threadIdx.x — maps each thread to a unique array element.

Q5. How does the GPU achieve parallelism?

Through SIMT (Single Instruction Multiple Threads) execution. Threads are grouped into warps of 32 that execute the same instruction simultaneously on different data. With 40 SMs and multiple active warps per SM, the T4 can have thousands of threads in flight at once. When a warp stalls on memory, the SM instantly switches to another ready warp (zero-cost latency hiding).

Q6. What is CUDA memory hierarchy and its significance?

1. Registers (fastest, per-thread). 2. Shared memory (48 KB/block, ~100× faster than global). 3. L1/L2 cache (automatic). 4. Global memory (15 GB, slowest). Task 5 showed memory transfers take 95% of

total pipeline time — using shared memory for reused data dramatically reduces global memory bandwidth pressure.

Q7. How is CUDA memory managed using cudaMalloc and cudaMemcpy?

cudaMalloc((void**)&d_ptr, size) allocates in GPU global memory. cudaMemcpy(dst, src, size, cudaMemcpyHostToDevice) copies over the PCIe bus. cudaMemcpy(dst, src, size, cudaMemcpyDeviceToHost) returns results. cudaFree(d_ptr) releases GPU memory. The programmer handles all allocation and deallocation explicitly — no garbage collection on the GPU.

Q8. What is the purpose of cudaDeviceSynchronize()?

CUDA kernel launches are asynchronous — the CPU returns immediately after launching. cudaDeviceSynchronize() blocks the CPU until all GPU work completes. Used in Task 2 to ensure printf output appears before the next config header is printed. For timing, cudaEventSynchronize(stop) is preferred as it is lighter-weight and more precise.

Q9. Explain the significance of warp size (32) in CUDA.

The warp is the fundamental scheduling unit — all 32 threads execute the same instruction per cycle. If threads branch (if/else), both paths execute with inactive threads masked (warp divergence), reducing efficiency. For best performance: align data to 128-byte boundaries (coalesced access), avoid branch divergence, and use multiples of 32 for block sizes to avoid partially filled warps.

Q10. How does the number of threads per block affect performance?

Task 6 showed kernel time was nearly identical for block sizes 64–512 (0.051–0.053 ms). For this memory-bound kernel, the bottleneck is memory bandwidth not thread scheduling. For compute-intensive kernels using shared memory, block size critically affects occupancy. Best practice: 128–256 threads/block for general workloads.

Q11. What is the purpose of using cudaEvent_t for timing?

cudaEvent_t provides GPU-side hardware timestamps accurate to microseconds. CPU timers are unreliable for GPU kernels because launches are asynchronous. cudaEventRecord(start) inserts a timestamp into the GPU command queue. cudaEventElapsedTime computes actual GPU time between two events, independent of CPU scheduling jitter.

Q12. What is the significance of checking cudaGetDeviceCount()?

cudaGetDeviceCount() determines how many CUDA GPUs are available. If it returns 0, all subsequent CUDA calls fail. It enables graceful fallback to CPU or GPU selection in multi-GPU systems based on totalGlobalMem or multiProcessorCount per device.

Q13. How does the grid size relate to the input data size?

numBlocks = (N + threadsPerBlock − 1) / threadsPerBlock ensures enough blocks for all N elements (ceiling division). For N=1,000,000 and 256 threads/block: 3,907 blocks. The if (idx < n) guard prevents the ~230 idle threads in the last block from writing out-of-bounds. Grid size scales linearly with N (up to 2,147,483,647 in x-dimension).

Q14. What happens when blocks finish in different orders?

Task 3 confirmed blocks may execute out of order. The GPU scheduler assigns blocks to SMs as they become available — there is no ordering guarantee between blocks. This is fundamental to CUDA: blocks must be fully independent. Inter-block dependencies require separate kernel launches with cudaDeviceSynchronize() between them.

Q15. What is the speedup formula and what does it represent?

Speedup = CPU Time / GPU Kernel Time. Task 4: 2.018/0.104 = 19.43×. Task 7 (N=100M): 249.584/4.558 = 54.76×. Task 8: 2.015/0.121 = 16.70×. Note: this is kernel-only speedup — if memory transfers are included, total GPU time for Task 5 (N=500K) is 2.686 ms, which may exceed CPU time for small one-shot computations.

Q16. Explain the difference between sequential, OpenMP, and CUDA execution models.

Sequential: single CPU core, 1 thread, no overhead, best for small logic-heavy tasks. OpenMP: #pragma omp parallel for distributes iterations across 2–128 CPU threads; Colab had 2 threads → ~1.4× speedup. CUDA: maps each iteration to a GPU thread; thousands of parallel threads → 54.76× speedup at N=100M. CUDA requires separate device memory and explicit transfers.

Q17. Why is GPU computing beneficial for large-scale scientific computing?

1) Massive parallelism: T4 has 2,560 cores vs 2–64 CPU cores. 2) High bandwidth: GPU VRAM ~320 GB/s vs CPU ~50 GB/s. 3) Energy efficiency: more GFLOPS/watt for parallel workloads. 4) Scalability: CUDA scales to newer GPUs automatically. Applications: molecular dynamics, weather simulation, deep learning, seismic imaging, genomics.

Q18. What are the limitations of GPU computing?

1) Memory transfer overhead: PCIe bus is slow (Task 5: transfers 19× slower than kernel). 2) Memory capacity: 15 GB GPU VRAM < typical RAM. 3) Programming complexity: explicit memory management required. 4) Warp divergence: branchy code runs poorly. 5) Small inputs: GPU overhead does not pay off for N < ~10,000. 6) Amdahl's law: sequential fraction limits maximum speedup.

Q19. How does CUDA support different hardware via compute capability?

Compute Capability (CC) versions CUDA hardware features. T4 is CC 7.5 (Turing): Tensor Cores for AI, RT Cores for ray tracing, INT8/INT4 inference. Code compiled for lower CC is forward-compatible. nvcc --arch=sm_75 generates PTX+SASS optimized for T4. Higher CC (8.0 Ampere, 9.0 Hopper) adds newer features.

Q20. What are best practices for writing efficient CUDA code?

1) Use 128–256 threads/block for good occupancy. 2) Ensure coalesced memory access (adjacent threads → adjacent addresses). 3) Use shared memory for frequently reused data. 4) Minimize H↔D transfers; keep data on GPU across multiple operations. 5) Avoid warp divergence. 6) Use cudaEvent_t for accurate timing. 7) Add CUDA_CHECK macros for all API calls. 8) Use cuBLAS/cuFFT/cuDNN — they outperform hand-written kernels.

8. Reflection Questions

R1. What surprised you most about GPU programming?

The most surprising finding was how memory transfers dominate over computation. Task 5 showed H→D + D→H transfers (2.555 ms) taking 19× longer than the kernel (0.131 ms). Intuitively one expects parallel computation to be the bottleneck, but the PCIe bus is the real limiting factor for many workloads. This changes the entire design philosophy toward keeping data resident on the GPU across multiple operations.

R2. What was the most challenging aspect of CUDA programming?

Understanding the thread indexing model was most challenging — computing the global thread ID (int idx = blockIdx.x * blockDim.x + threadIdx.x) requires thinking at two levels simultaneously. Additionally, the if (idx < n) boundary guard is easy to forget, causing silent out-of-bounds GPU memory access with no compiler warning — a class of bug that does not exist in sequential programming.

R3. How does the CUDA programming model compare to traditional programming?

Traditional programming is naturally linear — instructions execute sequentially with shared variables and predictable order. CUDA inverts this: thousands of threads run simultaneously executing the same function on different data. Traditional debugging tools do not work on GPU code. The programmer must reason about: which thread handles which data, inter-thread dependencies, and memory access patterns — a fundamentally different mental model.

R4. What improvements would you make to the code?

1) Unified Memory (cudaMallocManaged) to eliminate explicit H→D/D→H transfers. 2) CUDA Streams to overlap transfers and computation. 3) Shared memory tiling for more complex kernels. 4) CUDA_CHECK macros on all API calls for robust error handling. 5) Grid-stride loops so each thread processes multiple elements, improving GPU utilization for small inputs.

R5. How would you optimize the vector addition kernel?

1) Vectorized loads using float4 (load 4 elements per instruction). 2) Grid-stride loops (each thread processes multiple elements). 3) __ldg() for read-only data (load via texture cache). 4) Fused multiply-add (FMA) operations to increase arithmetic intensity. For this memory-bound kernel, the primary gain comes from maximizing memory bandwidth utilization rather than reducing compute instructions.

R6. What real-world applications benefit from CUDA programming?

1) Deep Learning: GPU matrix operations in PyTorch/TensorFlow (ResNet, BERT, GPT). 2) Medical Imaging: CT/MRI reconstruction and real-time segmentation. 3) Molecular Dynamics: Protein folding simulation (AlphaFold2). 4) Computational Finance: Monte Carlo option pricing. 5) Video Encoding: Real-time H.265 on YouTube/Netflix. 6) Climate Simulation: Weather forecasting models. 7) Genomics: GPU-accelerated GATK variant calling.

R7. How does CUDA handle thread synchronization?

__syncthreads() provides a block-level barrier — all threads in the block must reach this point before any proceeds, enabling safe shared memory use (write → sync → read). For global synchronization, blocks cannot sync during a kernel; separate kernel launches with cudaDeviceSynchronize() on the host achieve this. CUDA 9+ cooperative groups provide more flexible group-level synchronization primitives.

R8. What did you learn about parallel computing from this experiment?

1) Amdahl's Law: even 54.76× kernel speedup doesn't help if 50% of the app is sequential. 2) Granularity: N=100K gave 3.86× speedup; N=100M gave 54.76× — parallelism pays only with enough work. 3) Memory is often the bottleneck, not compute. 4) Correctness first: always verify GPU results against a CPU reference. 5) Hardware defines strategy: T4's 40 SMs, 48 KB shared memory, and 256-bit bus determine which optimization techniques to apply.

9. AI & HPC Case Study: Deep Learning Training on GPUs

9.1 Why GPUs Transformed AI

Deep learning's renaissance from 2012 onward was made possible almost entirely by GPUs. The key operation in training a neural network — matrix multiplication in forward and backward passes — maps perfectly to GPU architecture. A single forward pass of ResNet-50 involves billions of multiply-add operations; training over millions of images requires trillions. Before GPUs, this was computationally prohibitive on CPUs.

9.2 How CUDA Enables Deep Learning

Modern frameworks like PyTorch and TensorFlow use CUDA under the hood. When a user writes:

```
output = torch.matmul(A, B)    # dispatches to cuBLAS on GPU
loss.backward()                 # gradient computation via custom CUDA kernels
```

PyTorch dispatches these to highly optimized CUDA kernels — cuBLAS for matrix ops, cuDNN for convolutions. The Tesla T4's Tensor Cores (CC 7.5, Turing architecture) perform 4×4 FP16 matrix multiplications in a single clock cycle, delivering up to 65 TOPS — 65× faster than pure FP32 on the same GPU.

9.3 Large Language Model Training

Training GPT-3 (175B parameters) required 3,640 A100 GPUs for 34 days. Each A100 (CC 8.0) delivers 312 TFLOPS of tensor-core performance. Without GPU-accelerated CUDA, this would take centuries on CPUs. Techniques built on CUDA fundamentals from this experiment:

- Tensor parallelism: split weight matrices across GPUs
- Pipeline parallelism: different GPU nodes handle different transformer layers
- Mixed precision (FP16/BF16): halve memory, double throughput via Tensor Cores
- Gradient checkpointing: recompute activations to reduce VRAM usage

9.4 Connection to This Experiment

The vector addition kernel (Task 4, 19.43× speedup) is conceptually identical to the element-wise operations in a Transformer attention layer: multiply, add, activate. At production scale — weight matrices of size 4096×4096 per attention head, 96 heads — these speedups compound to make trillion-parameter models trainable. This experiment demonstrates the foundational CUDA building blocks on which all modern AI infrastructure is built: kernel launches, thread indexing, memory management, and event-based timing.